

**ARTIFICIAL INTELLIGENCE
CLASSIFICATION REGRESSION**



KHEN MUHAMMAD CAHYO

244107023002

TI-21

**STATE POLYTECHNIC OF MALANG
INFORMATION TECHNOLOGY DEPARTMENT**

1. K-NEAREST NEIGHBORS (KNN)

1. K- Nearest Neighbors (KNN)

```
[1] import numpy as np
    from sklearn.neighbors import KNeighborsClassifier

    # Data training: Heavy (kg), Tall (cm)
    X_train = np.array([[55, 160], [58, 162], [60, 165], [63, 170], [65, 172], [70, 175]])
    y_train = np.array(["P", "P", "P", "L", "L", "L"]) # Label Class

    # Data new Which want to classified
    X_new = np.array([[61, 167]])

    # Make model KNN with K = 3
    knn = KNeighborsClassifier(n_neighbors=3)
    knn.fit(X_train, y_train)

    # Prediction class For data new
    y_pred = knn.predict(X_new)
    print(f>Data {X_new.tolist()} classified as: {y_pred[0]}")
```

➡ Data [[61, 167]] classified as: P

The implementation of K-Nearest Neighbors (KNN) in the code aims to classify the gender of a person based on weight (kg) and height (cm) data. The training dataset (X_{train}) contains pairs of weight and height values, each labeled “P” (female) or “L” (male) in y_{train} . The KNN model was built with a value of $k=3$, which means that the model will consider the 3 nearest neighbors of the new data ($X_{\text{new}} = [[61, 167]]$) to determine its class. After the model was trained with the training data using `knn.fit`, prediction was performed with `knn.predict`, and the results showed that the new data was classified as the majority class of its 3 nearest neighbors.

2. NAÏVE BAYES

2. Naive Bayes

```
[5] from sklearn.naive_bayes import GaussianNB

# Data training: Heavy (kg), Tall (cm)
X_train = np.array([[55, 160], [58, 162], [60, 165], [63, 170], [65, 172], [70, 175]])
y_train = np.array(["P", "P", "P", "L", "L", "L"])

# Data new Which want to classified
X_new = np.array([[61, 167]])

# Initialization model Naive Bayes
nb = GaussianNB()

# Train the model
nb.fit(X_train, y_train)

# Prediction
y_pred = nb.predict(X_new)

print(f"Data {X_new.tolist()} classified as: {y_pred[0]}")
```

➡ Data [[61, 167]] classified as: P

The Naive Bayes algorithm implementation in the code is used to classify gender based on weight and height attributes. The training dataset (X_train) contains weight and height feature values, while the target labels (y_train) indicate the classes “P” (female) and “L” (male). The model used is Gaussian Naive Bayes (GaussianNB), which assumes that the input features follow a normal (Gaussian) distribution. After the model is trained using nb.fit, new data (X_new = [[61, 167]]) is predicted with nb.predict, and the prediction results show which class is most likely based on the probability of each feature to the learned class.

3. SUPPORT VECTOR MACHINE

3. Support Vector Machine (SVM)

```
from sklearn.svm import SVC
|
# Data training: Heavy (kg), Tall (cm)
X_train = np.array([[55, 160], [58, 162], [60, 165], [63, 170], [65, 172], [70, 175]])
y_train = np.array(["P", "P", "P", "L", "L", "L"]) # Label Class

# Data new Which want to classified
X_new = np.array([[61, 167]])

# Initialization model SVM with kernel linear
svm = SVC(kernel='linear')

# Train the model
svm.fit(X_train, y_train)

# Prediction
y_pred = svm.predict(X_new)

print(f"Data {X_new.tolist()} classified as: {y_pred[0]}")
```

➡ Data [[61, 167]] classified as: P

The Support Vector Machine (SVM) implementation in this code is used to classify gender based on weight and height features. The training data (X_{train}) contains pairs of weight and height values, with class labels “P” (female) and “L” (male) in y_{train} . The SVM model is initialized using a linear kernel (`kernel='linear'`), which means the algorithm will search for a line (or hyperplane in higher dimensions) that separates the two classes by the largest margin. After the model is trained using `svm.fit`, the new data ($X_{\text{new}} = [[61, 167]]$) is classified with `svm.predict`, and the result shows which side of the dividing line the data falls on, thus determining the most appropriate class.

4. DECISION TREE

4. Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

# Data training: Heavy (kg), Tall (cm)
X_train = np.array([[55, 160], [58, 162], [60, 165], [63, 170], [65, 172], [70, 175]])
y_train = np.array(["P", "P", "P", "L", "L", "L"]) # Label Class

# Data new Which want to classified
X_new = np.array([[61, 167]])

# Initialization model Decision Tree
dt = DecisionTreeClassifier()

# Train the model
dt.fit(X_train, y_train)

# Prediction
y_pred = dt.predict(X_new)

print(f"Data {X_new.tolist()} classified as: {y_pred[0]}")
```

➤ Data [[61, 167]] classified as: P

The Decision Tree implementation in this code aims to classify gender based on weight and height data. The training dataset (X_{train}) consists of pairs of weight and height values, with class labels “P” (female) and “L” (male) in y_{train} . The `DecisionTreeClassifier` model builds a decision tree structure based on the features that most effectively divide the data into these classes, by selecting split points based on metrics such as Gini impurity or entropy. After the model is trained with `dt.fit`, new data ($X_{\text{new}} = [[61, 167]]$) is predicted with `dt.predict`, where the model traverses the tree from the root to the leaves to determine which class is the best fit based on the split criteria established during training.

5. RANDOM FOREST

```
5. Random Forest

from sklearn.ensemble import RandomForestClassifier

# Data training: Heavy (kg), Tall (cm)
X_train = np.array([[55, 160], [58, 162], [60, 165], [63, 170], [65, 172], [70, 175]])
y_train = np.array(["P", "P", "P", "L", "L", "L"]) # Label Class

# Data new Which want to classified
X_new = np.array([[61, 167]])

# Initialization model Random Forest with 100 tree
rf = RandomForestClassifier(n_estimators=100)

# Train the model
rf.fit(X_train, y_train)

# Prediction
y_pred = rf.predict(X_new)

print(f"Data {X_new.tolist()} classified as: {y_pred[0]}")

Data [[61, 167]] classified as: P
```

The Random Forest implementation in this code is used to classify gender based on weight and height features. The training dataset (X_train) consists of pairs of weight and height values, with class labels “P” (female) and “L” (male) in y_train. The RandomForestClassifier model was formed from 100 decision trees (n_estimators=100) each trained on a random subset of the data and features, in order to produce a more stable and accurate model through the principle of majority voting. After training with rf.fit, new data (X_new = [[61, 167]]) is predicted using rf.predict, where the final result is the voting result of all trees in the forest, thus improving the reliability of the prediction compared to a single decision tree.

Here the link of my classification regression implementation results on Google Colab:

<https://colab.research.google.com/drive/1kk6slHH7etW2QLaxncqNRQoDR0YTlJBc?usp=sharing>